Rapport Projet transverse I - Rapport final (examen)

Projet: système de vente des produits de pâtisserie et gâteaux en ligne

Par: Dariia Zhuravel, Mihaela Satcovschi

Table de matières:

I. Introduction	2
II. Rappel du projet	2
III. Use case	3
IV. Analyse des objectifs (final)	4
V. Analyse des exigences (final - inclus les cas d'utilisation principaux)	5
VI. Schéma conceptuel (entité-relation) de la BDD (final)	6
VII. Schéma logique (phpMyAdmin) de la BDD (final)	8
VIII. Algorithmes (pseudo-code)	9
IX. Architecture	12
X. Implémentation client - serveur	14
XI. Récupération des données de test	15
XII. Résultats	18
XIII. Gestion de projet (sprints + outils)	18
XIV. Conclusion	19

I. Introduction

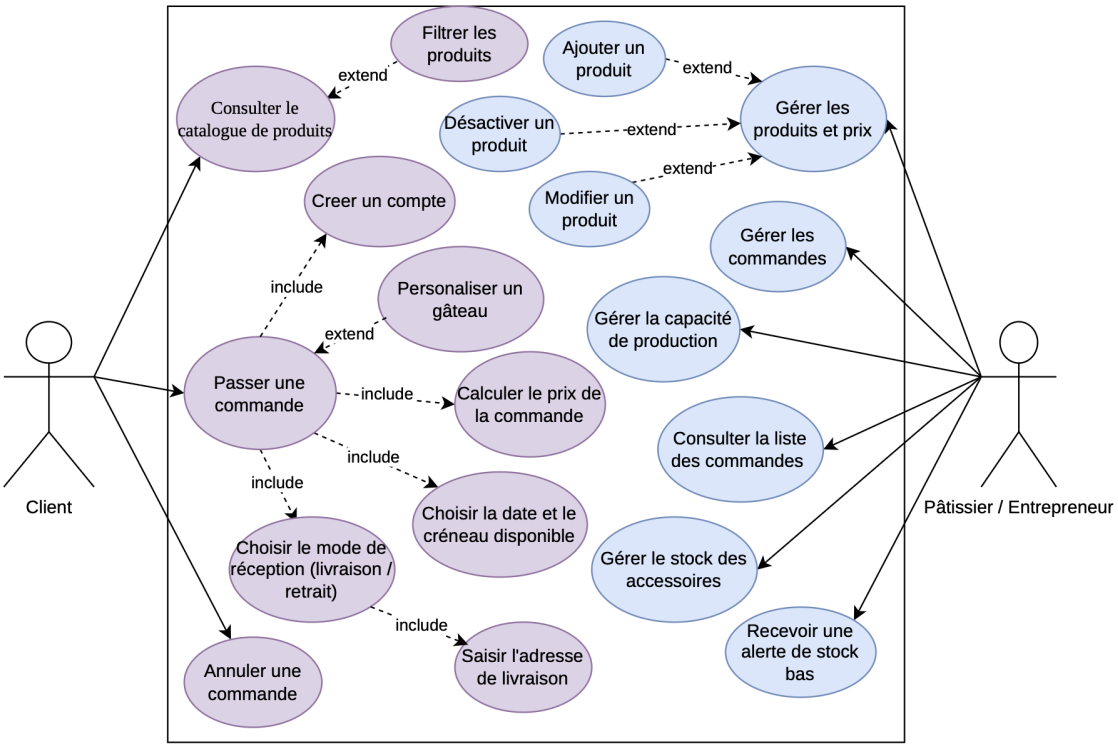
Pour le Projet Transverse 1, nous avons choisi le thème de la vente en ligne. Plus précisément, nous avons décidé de nous inspirer de la vente en ligne de gâteaux et autres pâtisseries pour créer un système unique, entièrement adapté au choix et à la personnalisation des gâteaux. Notre système s'adressera à deux publics : aux clients, afin qu'ils puissent passer des commandes, et aux entrepreneurs, afin qu'ils puissent recevoir ces commandes et les gérer.

II. Rappel du projet

Dans le contexte d'une petite entreprise, la gestion des commandes se fait souvent de manière informelle (messages, appels, notes), ce qui peut entraîner des erreurs, un manque de visibilité sur les demandes en cours, ainsi que des difficultés à organiser la production et à gérer les stocks (ici, nous nous concentrons spécifiquement sur les produits complémentaires (emballages, bougies, décorations, etc.) et non sur les ingrédients de fabrication). L'objectif de notre système est donc de centraliser ces informations dans une seule plateforme, afin de faciliter la prise de commande, de réduire les oublis et d'améliorer l'organisation au quotidien.

L'idée principale est de créer un site web grâce auquel les clients pourront découvrir la gamme de produits, les designs de base possibles, les garnitures et les prix. Nous souhaitons également ajouter une interface de commande où le client pourra choisir la date qui lui convient, vérifier les créneaux disponibles, personnaliser un gâteau (poids, garniture, design, inscription, extras), ajouter des commentaires à la commande, choisir la livraison ou le retrait et obtenir le calcul du prix. Du côté de l'entrepreneur, une interface d'administration permet de gérer les produits disponibles, les options, les prix, etc. Il disposera aussi d'une base de données contenant toutes les commandes, les stocks(au moins pour les accessoires) et les produits avec prix.

III. Use case



Le diagramme de cas d'utilisation représente les interactions d'un Client et d'un Pâtissier / Entrepreneur dans notre système de commande de pâtisseries en ligne.

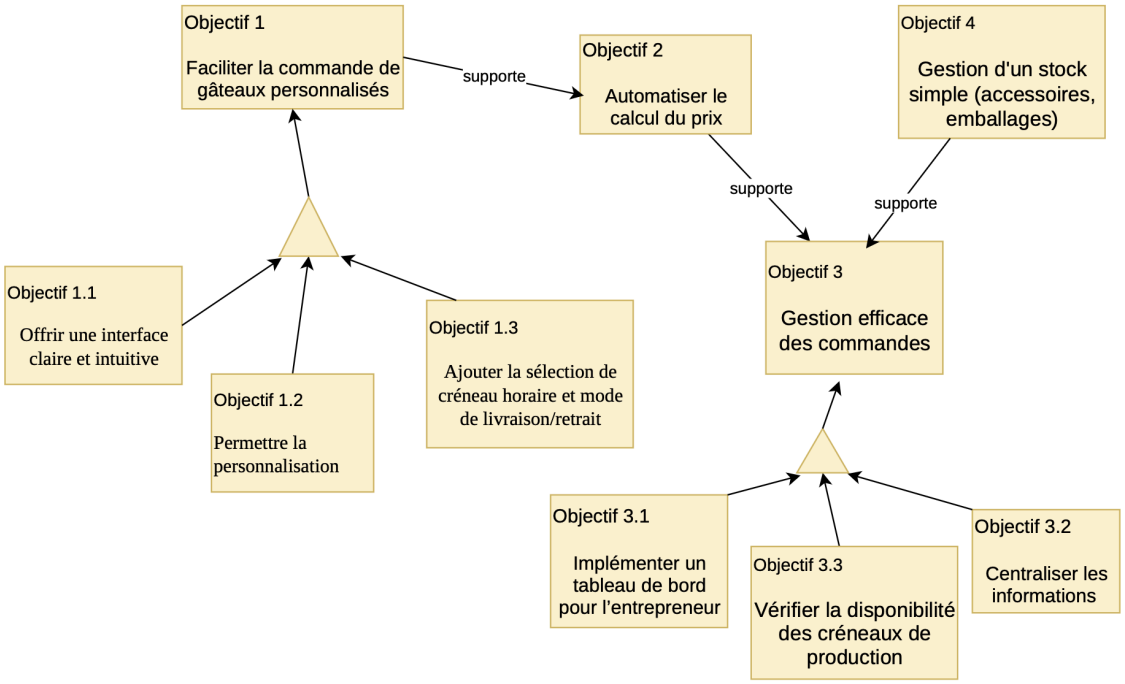
Du côté du client, les principales fonctionnalités sont la consultation du catalogue de produits, le passage d'une commande ou l'annulation. De plus, le cas d'utilisation «filtrer les produits» est associé à «consulter le catalogue de produits» par une relation «extend», car le client peut consulter le catalogue sans nécessairement appliquer de filtre.

Le cas d'utilisation « passer une commande » occupe une place centrale dans le diagramme, car plusieurs actions en dépendent directement, notamment le calcul du prix total, le choix d'une date et d'un créneau horaire disponible, ainsi que la sélection du mode de réception, par livraison ou par retrait. Ces actions sont représentées comme liées au processus principal par une relation « include », puisqu'elles sont nécessaires à la finalisation de la commande. La création de compte est également liée à ce processus par une relation « include », dans la mesure où, dans notre système, elle est considérée comme nécessaire pour faciliter la gestion et le suivi des commandes par l'utilisateur. En revanche, la personnalisation d'un gâteau correspond à une possibilité supplémentaire représentée par une relation « extend », car elle n'intervient

pas dans toutes les commandes, mais seulement lorsque le client souhaite adapter le produit à ses besoins.

Du côté du pâtissier ou entrepreneur, le système offre des fonctions de gestion avancée de l'offre et des opérations. Il peut gérer les produits et leurs prix, et les cas d'utilisation « ajouter un produit », « modifier un produit » et « désactiver un produit » sont reliés à cette fonction par des relations « extend », puisqu'ils correspondent à des actions spécifiques réalisées selon le besoin, et non à des étapes systématiques de chaque gestion. Le professionnel dispose aussi d'une gestion des commandes, incluant la consultation de la liste des commandes et l'annulation d'une commande si nécessaire. Dans ce cadre, « annuler une commande » peut également être représenté par une relation « extend », car cette action n'intervient que dans des situations particulières et ne fait pas partie du déroulement normal de chaque gestion de commande. Enfin, il gère le stock des accessoires, la capacité de production, et reçoit des alertes en cas de stock bas, ce qui lui permet d'anticiper les besoins et de garantir la disponibilité des produits pour les clients.

IV. Analyse des objectifs (final)



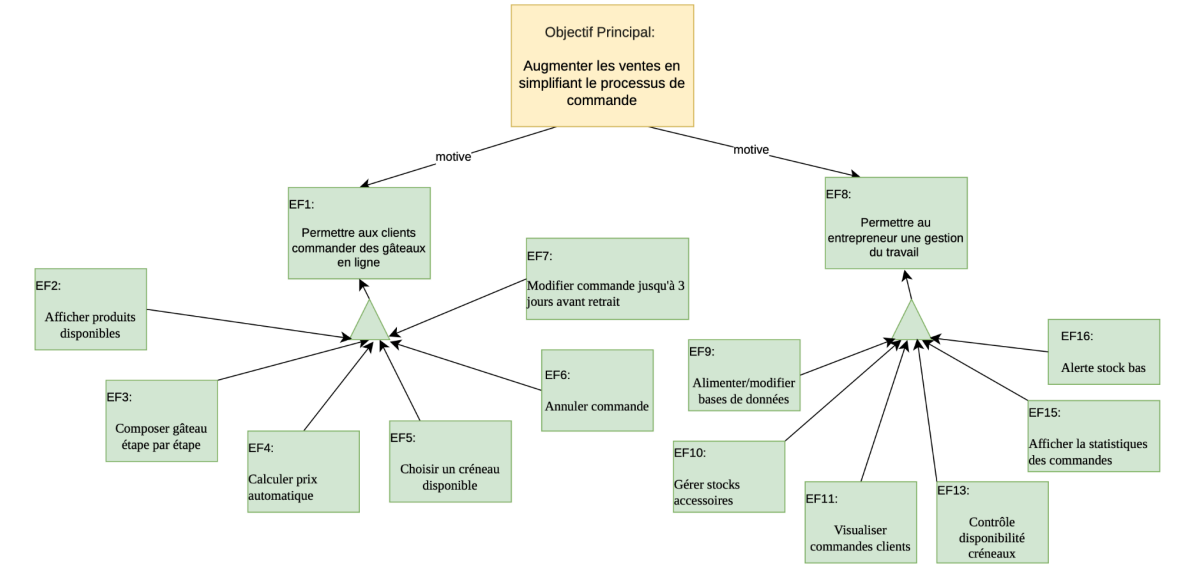
Ce schéma présente des objectifs de notre système de commande de gâteaux personnalisés. L'Objectif 1 vise à faciliter la commande de gâteaux personnalisés en mettant l'accent sur l'expérience client : il s'agit d'offrir une interface claire et intuitive (1.1), de permettre la personnalisation des produits (1.2) ainsi que d'ajouter la sélection du créneau horaire et du mode de livraison ou de retrait (1.3).

L'Objectif 2 concerne l'automatisation du calcul du prix, ce qui vient soutenir la gestion des commandes en rendant la tarification plus rapide, cohérente et fiable.

L'Objectif 3 porte sur une gestion efficace des commandes avec trois sous-objectifs complémentaires : implémenter un tableau de bord pour l'entrepreneur (3.1), centraliser les informations (3.2) et vérifier la disponibilité des créneaux de production (3.3) afin d'aligner les demandes clients avec les capacités de production.

Enfin, l'Objectif 4 prévoit la gestion d'un stock simple d'accessoires et d'emballages, ce qui vient également soutenir le bon déroulement du processus de gestion.

V. Analyse des exigences (final - inclus les cas d'utilisation principaux)



Ce schéma décrit les principales exigences fonctionnelles de notre application, à partir d'un objectif principal: augmenter les ventes en simplifiant le processus de commande. Pour cela, nous prévoyons de développer une application permettant à la fois aux clients de commander des gâteaux en ligne (EF1) et à l'entrepreneur de gérer efficacement son travail (EF8).

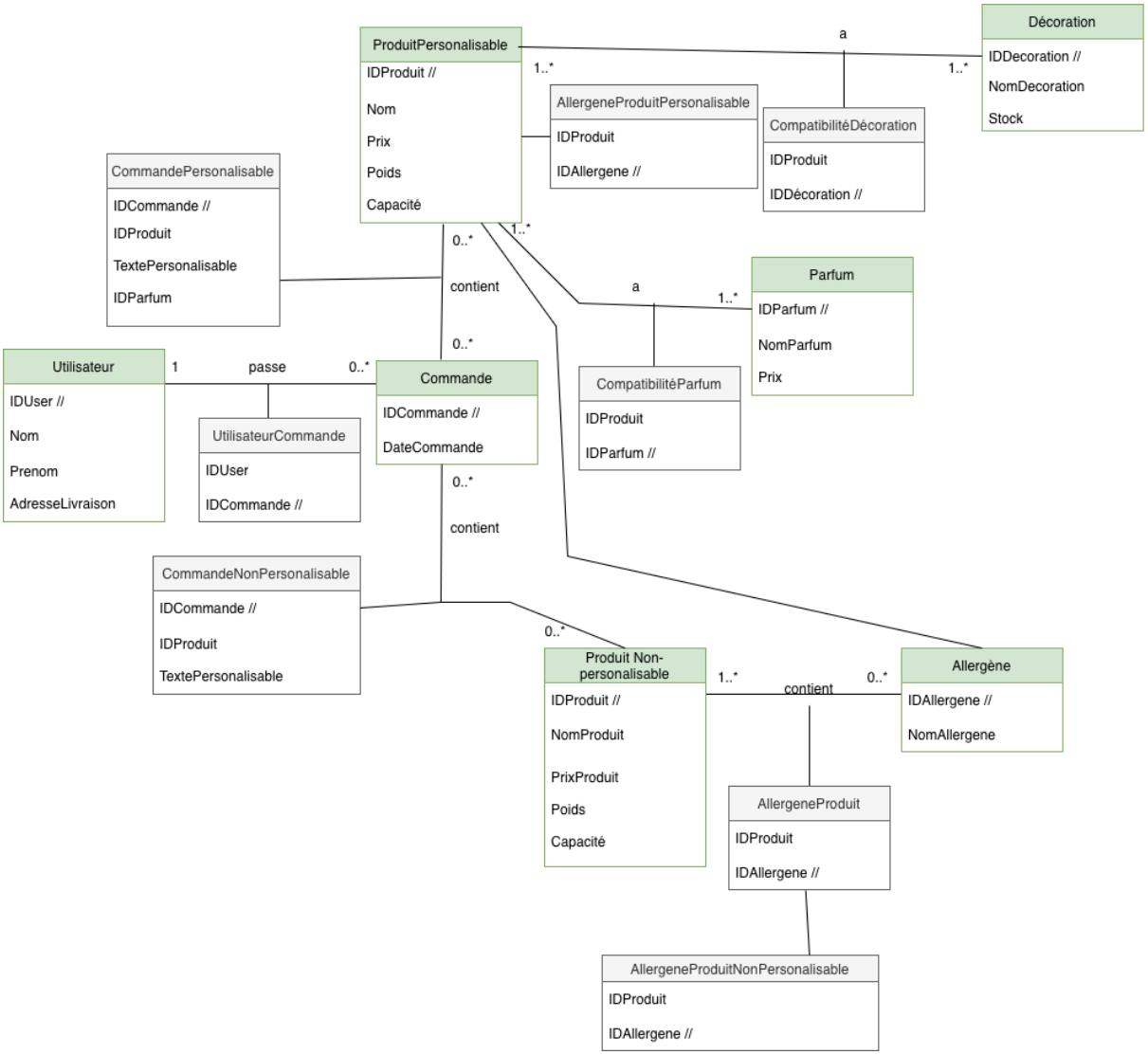
Autour de cet objectif central, plusieurs fonctions orientées client sont mises en avant : afficher les produits disponibles (EF2), composer un gâteau étape par étape (EF3), calculer automatiquement le prix (EF4), choisir un créneau disponible (EF5), annuler une commande (EF6) ou encore la modifier jusqu'à trois jours avant le retrait (EF7), ainsi que visualiser les commandes clients (EF11).

Du côté de l'entrepreneur, le schéma prévoit des fonctionnalités de gestion interne telles qu'alimenter ou modifier les bases de données (EF9), gérer les stocks d'accessoires (EF10), déclencher une alerte de stock bas (EF16), afficher des statistiques sur les commandes (EF15) et contrôler la disponibilité des créneaux (EF13), ce qui garantit une exploitation plus structurée et efficace de l'activité.

En complément de ces aspects fonctionnels, on a défini des exigences non-fonctionnelles qui garantissent la qualité globale de l'application, pour le client comme pour l'entrepreneur :

- Utilisabilité : l'application doit être simple à utiliser et lisible sur ordinateur et mobile.
- Sécurité : accès protégé par mot de passe, droits séparés client/admin, validation des champs saisis.
- Fiabilité : l'application ne doit pas planter en cas d'erreur ; elle affiche des messages clairs si une action échoue.
- Performance : les pages principales doivent se charger rapidement en utilisation normale.

VI. Schéma conceptuel (entité-relation) de la BDD (final)



Ce schéma conceptuel n'a pas été modifié depuis le dernier rendu, par manque des observations concernant le besoin d'un changement de sa structure. Voici sa structure et explications:

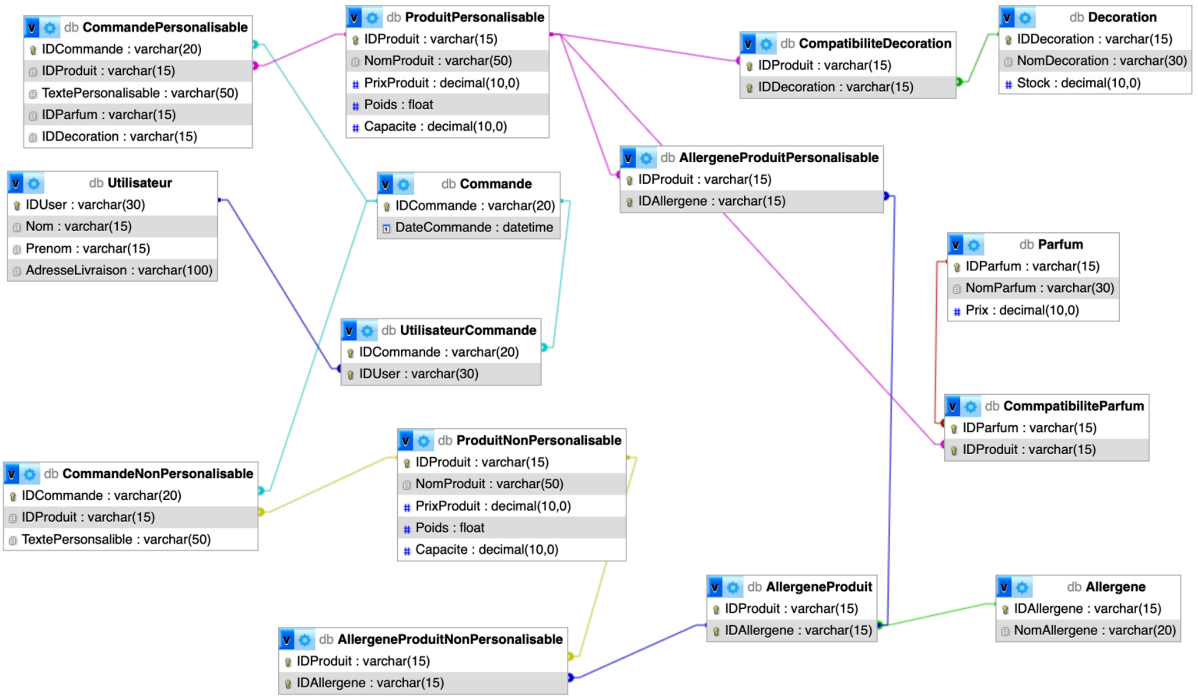
Suite à un rendez-vous avec l'assistant, on a refait la structure de notre base de données afin d'avoir une structure correcte, logique et fonctionnelle. Cette version ne correspond pas à la version du CheckPoint 1, donc tous les détails de la base de données (création et tables existantes), vont être détaillés dans le point suivant, concernant le schéma logique.

On a également fait quelques modifications qui peuvent être considérées comme redondantes, mais elles étaient faites pour la partie de récupération de données par le programme. On parle ici des tables Commandes, CommandeNonPersonalisable et CommandePersonalisable. La raison pour laquelle ces tables sont toutes séparées et pas mises dans une grande table est parce que si la table Commande serait construite d'utiliser comme clé étrangère l'attribut "IDProduit" de la table ProduitPersonalisable

et l'attribut "IDProduit" de la table "ProduitNonPersonalisable", alors pour une commande qui ne contient qu'un type de produit (produit personalisable ou produit non-personalisable), le tuple devrait être NULL pour le type de produit qui ne serait pas présent, et dans la lecture de la base de données avec python, les tuples de la base de données ne seraient pas clairement comprises et traitées. C'est bien pour ça qu'on a créé la table Commande avec uniquement les attributs "IDCommande" et "DateCommande" (-> par cette pratique on ne lie plus la table Commande aux 2 tables de produits) et les tables CommandePersonalisable et CommandeNonPersonalisable où dans les 2 cas, l'attribut "IDProduit" provient d'une seule table uniquement.

Par le même raisonnement, on a modifié la table AllergeneProduit, qui était dans la même construction: l'attribut "IDProduit" provient des 2 tables. Par conséquent, on a divisé cette table en 2 autres tables supplémentaires, conformes aux type de produit avec les allergènes concernés.

VII. Schéma logique (phpMyAdmin) de la BDD (final)



Par principe du fait que le schéma conceptuel n'a pas changé depuis le dernier rendu, le schéma logique n'a pas changé non plus. Voici le screenshot de l'écran du schéma logique depuis phpMyAdmin.

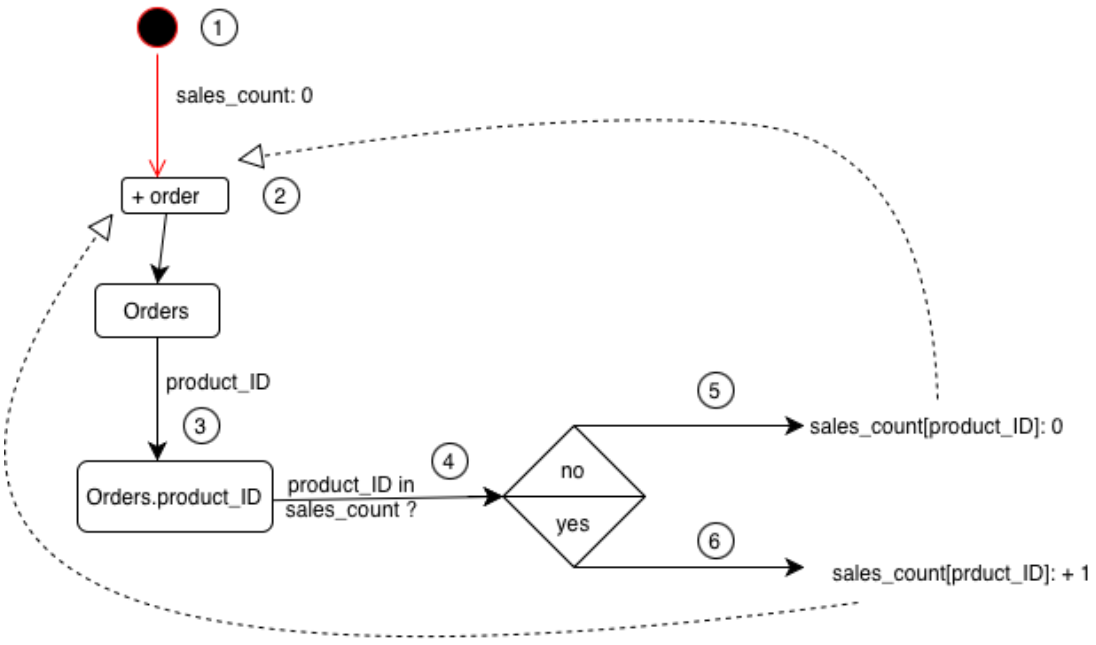
VIII. Algorithmes (pseudo-code)

Suite au rendu du CheckPoint 2, on a reçu des retours expliquant quelles devraient être les modifications qui à amener concernant nos algorithmes. Par conséquent, on a 2 algorithmes mises en place pour le système fonctionnel de vente en ligne: un premier algorithme qui tenait compte des produits commandés (et des décorations ajoutées pour les produits de la commande) afin de pouvoir ajouter des étiquettes avec le texte “Bestseller” sur les produits qui ont été commandés un certain nombre des fois. Un deuxième algorithme a été fait pour créer des recommandations des produits pour les utilisateurs qui ont fait un certain nombre des commandes selon le type d’allergènes présent dans les produits commandés.

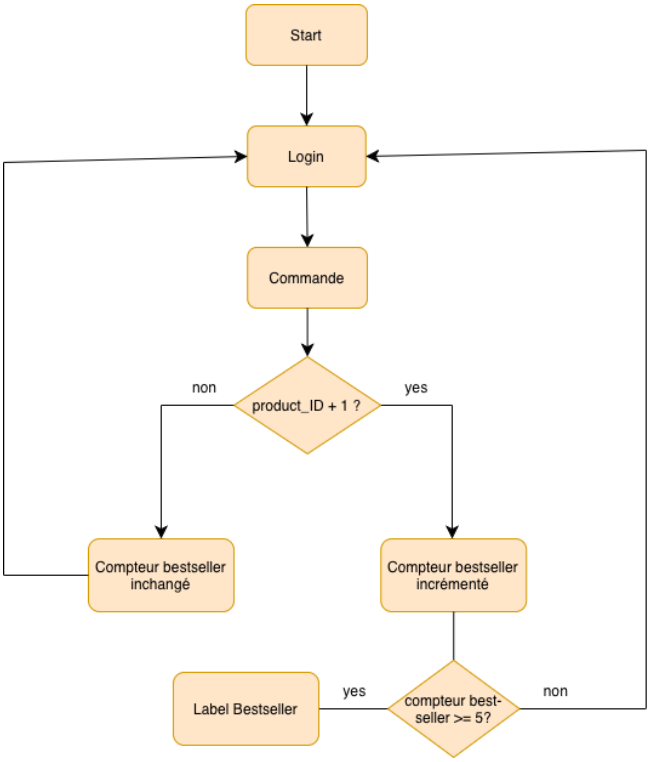
En ce qui concerne le premier algorithme, voici le pseudocode:

```
FOR product in range( orders ):
        sales_counter ← 0
                IF order. product_ID = product THEN
                        counter ← sales_count + 1
                END IF
        IF sales_count  ≥ 5 THEN
                product.status ← "Bestseller"
        END IF
END FOR
```

Diagramme UML correspondant:



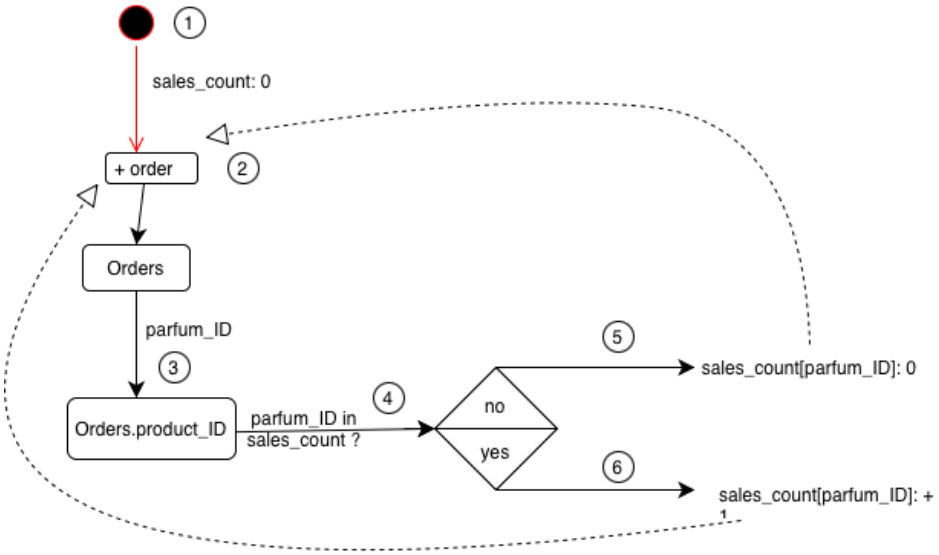
Flowchart concernant:



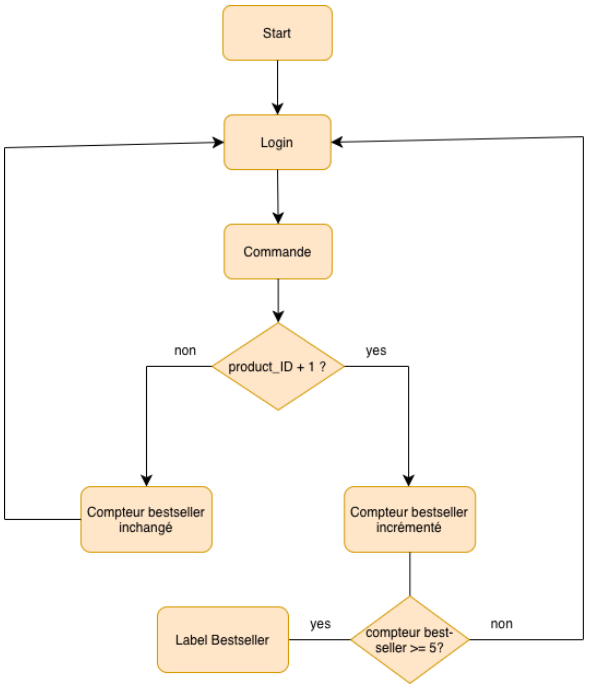
Voici le pseudo-code l'algorithme calculant les produits bestsellers (on va utiliser la convention que pour qu'un produit soit un bestseller, il faut que au moins 5 personnes l'ont commandé).

```
FOR parfum in range( orders ):
        sales_counter ← 0
                IF order. parfum_ID = parfumTHEN
                        counter ← sales_count + 1
                END IF
        IF sales_count  ≥ 5 THEN
                parfum.status ← "Bestseller"
        END IF
END FOR
```

Diagramme UML correspondant;

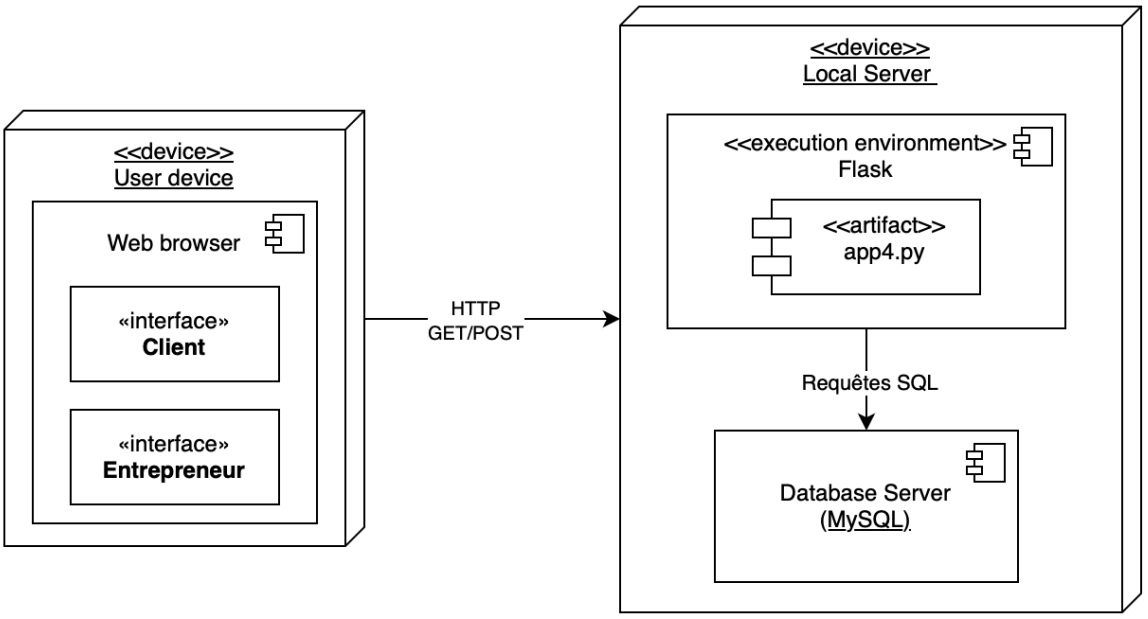


Flowchart correspondant:



En ce qui concerne l'algorithme de recommandations personnalisées pour les utilisateurs du système, en utilisant les allergènes d'un produit pour recommander un produit contenant exactement les mêmes allergènes, selon la recommandation des enseignants, par manque de temps et par besoin de prioriser l'implémentation du mode entrepreneur qui offre des fonctionnalité de contrôle et supervision sur les produits et les commandes créent.

IX. Architecture



La figure présente le schéma architectural de notre système web. Ce diagramme illustre l'emplacement physique des principaux composants de l'application et les interactions entre eux.

Dans notre cas, le système se compose de deux nœuds principaux : l'appareil de l'utilisateur et le serveur d'application local.

Le nœud User Device représente l'appareil de l'utilisateur, sur lequel l'accès au site web s'effectue via un navigateur web. En fonction du rôle de l'utilisateur, le système fournit une interface client ou entrepreneur, mais les deux types d'utilisateurs travaillent via le même site web.

La communication entre l'appareil de l'utilisateur et le serveur de l'application s'effectue via le protocole HTTP/HTTPS à l'aide des méthodes GET et POST. À la réception d'une requête, Flask la traite, interroge la base de données si nécessaire et renvoie à l'utilisateur la réponse correspondante sous la forme d'une page web.

Le nœud Local Server contient les principaux composants logiciels du système. Le framework Flask est utilisé comme environnement d'exécution ; il traite les requêtes HTTP provenant du navigateur de l'utilisateur. Au sein de l'environnement Flask, l'application principale app4.py est déployée ; elle met en œuvre la logique métier du

système, le traitement des requêtes utilisateur, la gestion des données et l'interaction avec la base de données.

Le serveur local héberge également une base de données MySQL dans laquelle sont stockées les données relatives aux utilisateurs, aux entrepreneurs, aux services et toute autre information nécessaire au fonctionnement du système. L'interaction entre l'application app4.py et la base de données s'effectue au moyen de requêtes SQL (SELECT, INSERT, UPDATE et DELETE).

Étant donné que le développement et les tests du système ont été effectués sur un seul ordinateur, l'environnement Flask, l'application app4.py et la base de données MySQL sont physiquement hébergés sur un seul serveur local.

Structure du projet

La structure du projet reflète l'organisation des fichiers et des répertoires de l'application web. Le fichier principal du système est app4.py, qui contient la logique côté serveur de l'application.

Le fichier .env sert à stocker les paramètres de configuration et les variables d'environnement nécessaires au fonctionnement de l'application.

Le répertoire « templates » contient les modèles HTML utilisés pour générer les pages web.

Le répertoire « static » est destiné à stocker les ressources statiques de l'application. Il comprend le fichier de styles CSS, ainsi que les images des produits utilisées lors de l'affichage de l'interface web.

```
project /
|-- app4.py
|-- .env
|-- templates /
|   |-- account.html
|   |-- add_product.html
|   |-- catalogue. html
|   |-- choose_prod.html
|   |-- entrepreneur_dashboard.html
|   |-- entrepreneur_login.html
|   |-- entrepreneur_orders.html
|   |-- home 1.html
|   |-- new_account.html
|   |-- new_product.html
|   |-- non_product_detail.html
```

```
|   |-- npers_prod.html
|   |-- orders.html
|   |-- pers_prod.html
|   |-- product_detail.html
|   |-- profile.html
|-- static/
|   |-- style.css
|   |-- home1.png
|   |-- redcake.png
|   |-- whitecake.png
|   |-- ...
```

X. Implémentation client - serveur

L'implémentation client - serveur du système de vente en ligne se divise en 3 composants:
client: le browser web de l'utilisateur, qui représente les pages HTML et CSS
serveur: l'application Flask écrite avec Python
base de donnés: créée à l'aide du container Docker et le site phpMyAdmin

Dans cette implémentation, le client (via le browser) envoie une requête HTTP (ou HTTPS) au serveur Flask. Le serveur Flask va envoyer une requête SQL à la base de données qui cherchent les données dans une table. Si la requête SQL arrive à récupérer les données de la base de données avec succès, le serveur Flask va amener l'utilisateur à une autre page HTML correspondante à la réponse attendue de l'utilisateur.

Un exemple concret représente l'interaction d'un utilisateur avec le système: l'utilisateur se retrouvent dans la page url: http://127.0.0.1:5000/account et il veut se connecter à son compte déjà existant dans la base de données, et il est capable d'introduire son username qui pourra être traité par le système, via la page HTML qui offre cette fonctionnalité (la page **account.html**). Une fois que l'utilisateur clique sur "Continue", une requête HTTP va être envoyée au serveur Flask qui s'occupe du traitement de l'input de l'utilisateur par la route **/account** du fichier **app4.py** qui evoie des requêtes SQL à la base de données. La base de données va effectuer la requête SQL et ensuite va donner comme réponse au serveur Flask, le serveur Flask va envoyer l'utilisateur dans la prochaine page HTTP (qui ici est **/profile**) et les résultats relatifs à l'utilisateur et les afficher dans l'interface utilisateur par la page **profile.html**.

Pour le composant client on comprend que le client a les privilèges suivants: afficher le catalogue, afficher les informations relatives à un produit, afficher les informations relatives à un compte utilisateur, créer des nouvelles comptes utilisateur, créer des nouvelles commandes, envoie des requêtes GET (collecte des certaines données) et POST (publication des certaines données).

Pour le composant serveur on comprend les privilèges suivants: envoie des requêtes HTTP, vérification de l'authentification des utilisateurs, envoie des requêtes SQL, calcul du profil "bestseller", envoie des pages HTML aux clients, tout via des routes différentes qui offrent un service distinct pour les utilisateurs du système.

La communication avec la base de données se fait via les lignes de code:
conn = get_db_connection()
cursor = conn.cursor(dictionary = True)
et par des requêtes SQL: SELECT (lire des données), INSERT (création des données), COUNT(DISTINCT)) (uniquement pour le calcul des produits et parfums "bestsellers")

XI. Récupération des données de test

Pour expliciter la pratique de récupération des données de test, on va reprendre les schéma conceptuel de la base de données, afin d'avoir toutes les tables concernées.
La table *ProduitPersonalisable* (IDProduit // NomProduit, PrixProduit, Poids, Capacité):

- l'attribut 'IDProduit' est un nombre d'identification commençant par la lettre 'G' et suivi par 4 chiffres générés aléatoirement avec le site de génération des données fictives mockaroo.com . Le choix de la lettre est d'avoir une distinction entre les IDs des produits personnalisables et les IDs des produits non-personnalisables (les IDs vont commencer avec la lettre 'N').
- L'attribut 'NomProduit' contient des vrais noms des produits, afin d'enrichir l'expérience utilisateur et en prenant en considération le fait que le nom d'un produit va être affiché de façon explicite pour les utilisateur, utiliser des vrais noms des produit aide à simuler au plus possible un site fonctionnel de vente en ligne.
- Les données d'attributs PrixProduit, Poids et Capacité ont été insérées de façon manuelle, pour but d'avoir une cohérence et un réalisme aux données insérées: par exemple, pour un gâteau donné, avec la capacité de 4 personnes, le poids respectif va être de 0.3 kg et le prix de base va être 35 CHF.

La table *Décoration* (IDDecoration// NomDecoration, Stock):

- l'attribut 'IDDecoration' a été inséré à la main, le premier produit ayant l'ID 1010 et le dernier produit ayant l'ID 1028. Il n'y a pas eu un motif intentionnel derrière ce choix.
- L'attribut 'NomDecoration' contient des vrais noms des vrais types de décorations qu'on a trouvé se renseignant, avec le but d'offrir de la clarté aux client au moment du choix d'une décoration donnée.
- L'attribut 'Stock' contient des chiffres générées par le site mockaroo.com

La table *ProduitNonPersonalisable* (IDProduit // NomProduit, PrixProduit, Poids, Capacité):

- l'attribut 'IDProduit' est un nombre d'identification commençant par la lettre 'N' et suivi par 4 chiffres générés aléatoirement avec le site de génération des données fictives mockaroo.com . Le choix de la lettre est d'avoir une distinction entre les IDs des produits personnalisables et les IDs des produits non-personnalisables (les IDs vont commencer avec la lettre 'N').
- L'attribut 'NomProduit' contient des vrais noms des produits, afin d'enrichir l'expérience utilisateur et en prenant en considération le fait que le nom d'un produit va être affiché de façon explicite pour les utilisateur, utiliser des vrais noms des produit aide à simuler au plus possible un site fonctionnel de vente en ligne.
- Les données d'attributs PrixProduit, Poids et Capacité ont été insérées de façon manuelle, pour but d'avoir une cohérence et un réalisme aux données insérées: par exemple, pour un gâteau donné, avec la capacité de 4 personnes, le poids respectif va être de 0.3 kg et le prix de base va être 35 CHF.

La table *Allergene*(IDAllergene // NomAllergene):

- l'attribut 'IDAllergene' a été introduit de façon manuelle, car les données de cette table ont été introduites depuis le début de la conception de la base de données et par le fait que la table a été correcte dans son implémentation, on a gardé les données de cette table.
- l'attribut 'NomAllergene' a été introduit de façon manuelle, en se renseignant sur la liste des 14 allergènes en Suisse, afin de pouvoir associer les gâteaux à leur vrais allergènes documentés, afin d'offrir une expérience réaliste pour les utilisateurs du système.

La table *Parfum*(IDParfum // NomParfum, Prix):

- l'attribut 'IDParfum' a été partiellement généré partiellement avec le site mockaroo.com en utilisant la lettre 'P' ensuite une lettre ou chiffre générées aléatoirement, suivi par 4 autres chiffres générées aléatoirement.

- l'attribut ‘NomParfum’ a été recherché et inséré manuellement pour offrir des choix des parfums réalistes, car tous les autres éléments dans le choix d’un produit personnalisable, afin de garder une cohérence dans les données
- l’attribut ‘Prix’ a été remplis à la main

La table *AllergeneProduit*(IDProduit IDAllergene //):
- les données ont été générées de façon pertinente pour que pour un IDProduit, on va créer toutes les tuples avec les allergènes qu’un produit contient. Par exemple, pour le IDProduit **G0286** qui est récupéré depuis la table ProduitPersonalisable et dans la table AllergneProduit contient 4 tuples qui ont un allergène pour le IDProduit: G0286 et plus exactement, les IDAllergenes: A00001, A00100, A00101, qui sont retrouvés dans la table Allergène comme: oeuf, soja et lait.

La table *Utilisateur*(IDUser // Nom, Prénom, AdresseLivraison):
- toutes les données ont été entièrement générées aléatoirement avec le site mockaroo.com aux indications des enseignants.

La table *CommandePersonalisable*(IDCommande // IDProduit):
- toutes les données ont été créés aléatoirement en combinant les données déjà insérées

La table *CommandeNonPersonalisable*(IDCommande // IDProduit, TextePersonalisable, IDParfum, IDDecoration):
- toutes les données ont été créés aléatoirement en combinant les données déjà insérées

La table *CompatibilitéParfum*(IDParfum IDProduit //):
- toutes les données ont été créés aléatoirement en combinant les données déjà insérées

La table *CompatibilitéProduit*(IDDecoration IDProduit):
- toutes les données ont été créés aléatoirement en combinant les données déjà insérées

Toutes les autres tables ne sont que des combinaisons des données des autres tables, par le besoin de devoir relier des tables entre elles. Ces tables sont: AllergeneProduitPersonalisable, AllergeneProduitNonPersonalisable, Commande, UtilisateurCommande.

XII. Résultats

On juge qu'à présent, on a réussi à mettre en place un système fonctionnel qui permet d'effectuer des requêtes GET et POST via une interface interactive, intuitive et facile. Ce qu'on a amené de nouveaux depuis le CheckPoint 2 est le mode entrepreneur l'application de l'algorithme du calcul des produits et parfums "Bestseller".
Le mode entrepreneur donne une deuxième dimension au projet et l'interaction avec la base de données avec la création de nouveaux produits à la place des requêtes SQL est beaucoup plus intuitive et prend moins de temps.

XIII. Gestion de projet (sprints + outils)

Sprint 0: Lors de la première semaine de travail dans le projet, on s'est occupé des détails administratifs qui vont représenter la base pour la suite du projet semestriel. On s'est occupé en spécial avec la recherche du partenaire de projet, on a considéré tous les projets et on leur a analysé leur niveaux de faisabilité avant d'en choisir un. On a échangé des idées et on a recherché les données cruciales qui concernent les éléments de la base de données (comme les types de gâteaux et les produits de pâtisserie, les ingrédients utilisés). Au final, on a rendu le cahier des charges.

Sprint 1: Lors de cette semaine, on a effectué la répartition des tâches pour les parties composantes du Checkpoint 1 et on s'est préoccupé de l'apprentissage approfondi du fonctionnement des outils nécessaires pour le projet, tel que phpMyAdmin, Docker et Git. On a également créé la base de données "sur papier" (dans des petites tables afin d'avoir une vision globale sur la base de données) et on a établi les objectifs et exigences principales.

Sprint 2: Cette semaine on a mis en pratique l'ensemble des connaissances accumulées des outils, on a dédié la plupart de la semaine pour la rédaction du rapport, de la présentation et de la création de la base de données pour qu'elle soit fonctionnelle.

Sprint 3: Etablissement des décisions concernant les détails fonctionnels, division des tâches pour le rapport, établissement détails des chemins et designs backend et frontend

Sprint 4: Changement des certains choix des chemins et designs backend et frontend, partage des idées de complétion du rapport.

Sprint 5: Création du mode entrepreneur, affinement du catalogue des produits

Sprint 6: Affinement du frontend, implémentation de l'algorithme de calcul des produits et parfum "bestsellers"

XIV. Conclusion

A ce stade, on trouve que le système est complet et fonctionnel par le mode entrepreneur récemment introduit, qui donne beaucoup plus de contrôle et supervision sur les aspects du système, notamment sur les commandes et les produits, ainsi que l'intégration de l'algorithme bestseller qui anime le site, en lui donnant une interaction de l'utilisateur beaucoup plus vivante.

Demo:
https://drive.google.com/file/d/119umHeS8mQ3Jlht3N8vBJzJyoWzVRNHV/view?usp=share_link